Perception & Multi-Modal Sensing Specification

Technical Guide to YOLOv11 Detection, SGBM Disparity Depth, and Euclidean Object Tracking

METRIC / SCOPE	SPECIFICATION / TARGET
Subsystem: Multi-Modal Perception	Module Path: core_safar_logic/python_perception
Vision Model: Ultralytics YOLOv11n	Inference Resolution: 640x640 RGB
Depth Method: SGBM Stereo / Raycast	Latency Budget: < 15.0 ms

1. Subsystem Architecture & Vision Pipelines

The Perception Subsystem transforms raw sensor streams (RGB camera video, stereo pairs, or UE5 raycasts) into standardized 3D obstacle representations (`PerceptionObject`). It combines deep neural network object detection (YOLOv11), dense disparity depth mapping (SGBM), and multi-frame temporal tracking.

2. Sensor Modalities & Depth Formulations

- **YOLOv11 Tensor Inference:** Runs GPU/CPU tensor models on 640×640 frames in $8 - 15\text{ ms}$.
- **Stereo Metric Depth:** $Z = \frac{f \cdot B}{d}$ where f is focal length, B is baseline (0.20m), and d is pixel disparity.
- **Temporal Kalman Association:** Tracks object centroids across frames to estimate closing velocity $\dot{Z} = \frac{\Delta Z}{\Delta t}$.

3. Source Code Files Breakdown

FILE NAME	RESPONSIBILITY & ARCHITECTURAL WORK
python_perception/yolo_detector.py	Wraps Ultralytics YOLOv11 (`yolo11n.pt`). Ingests RGB frames, runs tensor inference, outputs bounding boxes with confidence scores.
python_perception/yolo_adapter.py	Converts raw COCO detection boxes into standardized SAFAR obstacle models. Normalizes classes (`person`, `car`, `motorcycle`, etc.).
python_perception/stereo_depth.py	Computes stereo disparity using Semi-Global Block Matching (SGBM). Generates dense metric point clouds and estimates obstacle distances.
python_perception/tracker.py	Implements `MultiObjectTracker`. Euclidean association across frames; maintains track ID persistence and calculates range rates.
python_perception/camera.py	OpenCV camera capture loop. Manages USB webcams, RTSP streams, or recorded MP4 dashcam videos with thread-safe buffers.
python_perception/service/daemon.py	Runs standalone perception server over UDP/TCP socket. Enables offloading vision inference to a dedicated GPU edge computer.
ue5_adapter/Source/GroundTruthPerception.h/.cpp	Raycasting perception provider in Unreal Engine 5. Executes forward multi-point line traces and sphere sweeps up to 50 meters.

4. Line-by-Line Code Walkthrough: YOLO & Depth Normalization

```
class YoloAdapter:
    def adapt_detections(self, raw_boxes: List[Dict], depth_map: np.ndarray) -> List[PerceptionObject]:
        objects = []
        for box in raw_boxes:
            cls_id, conf = box['class'], box['confidence']
            if conf < 0.45: continue  # Reject low-confidence noise
            u1, v1, u2, v2 = box['bbox']
            # Sample median metric depth inside bounding box ROI
            roi_depth = depth_map[int(v1):int(v2), int(u1):int(u2)]
            valid_depth = roi_depth[roi_depth > 0.5]
            z_m = float(np.median(valid_depth)) if len(valid_depth) > 10 else 99.0
            # Project lateral coordinate X using camera intrinsics
            x_m = ((u1 + u2) / 2.0 - self.cx) * (z_m / self.fx)
            objects.append(PerceptionObject(box['id'], cls_id, conf, FVector(x_m, 0.0, z_m)))
        return objects
```

Code Explanation: Line 5 filters out low-confidence detections ($< 45\%$). Lines 8-10 sample the median depth inside the bounding box ROI to reject background pixels. Line 12 projects the 2D bounding box center into metric 3D coordinates using camera intrinsics (f_x, c_x).